

Code-Layout, Readability and Reusability

Date	3 July 2026
Team ID	
Project Name	Personalized Networking Assistant
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes/No/Partial)	Remarks
1.	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Standard 4-space Python indentation across all files
2.	Proper File Structure	Files and folders are logically organized	Yes	app/models, app/routers, app/services, frontend/, tests/ — clear separation of concerns
3.	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	e.g. event_themes, valid_facts, fact_snippet, bio_snippet
4.	Function/Method Names	Functions are descriptively named	Yes	e.g. extract_event_themes(), fact_check_many(), generate_topics(), log_conversation()
5.	Code Comments	Inline and block comments explain logic	Partial	Module-level headers and comments on key logic (e.g. the grounding step in conversation.py); not every function has a full docstring
6.	Modular Design	Code is split into reusable functions/modules	Yes	Business logic split into 5 independent service modules, consumed by a single router
7.	No Redundant Code	Duplicate or unused code is removed	Yes	Fact-checking logic centralized in fact_check_many(), reused by two endpoints rather than duplicated

8.	Error Handling	Exceptions and errors are handled gracefully	Yes	fact_checker.py wraps Wikipedia calls in try/except with graceful fallback strings; generation handles missing/failed facts
----	----------------	--	-----	---

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1.	fact_checker.fact_check()	Python / Requests	Used by both /fact-check and /generate-conversation (via fact_check_many)	High
2.	Pydantic schemas (schemas.py)	Python / Pydantic	Reused across all router endpoints for request/response validation	High
3.	history_logger.log_conversation()	Python / JSON	Called by /generate-conversation; read by the frontend History view	Medium
4.	feedback_logger.log_feedback()	Python / JSON	Called by /feedback; data structure ready for a future feedback-history view	Medium
5.	topic_generator.generate_topics()	Python / Hugging Face Transformers	Core generation logic, single reusable entry point	Medium

Overall Code Quality Assessment:

Aspect	Rating(1-5)	Comments
Code Layout & Structure	4	Clear separation of concerns across models/routers/services/frontend
Readability	4	Descriptive naming throughout; could use more formal docstrings
Reusability	4	Core services are clean, independently testable functions with minimal coupling
Documentation / Comments	3	Module headers and key inline comments present, but sparse formal docstrings
Overall Score	4/5	